



50 *años*

CURSO DE PÓS-GRADUAÇÃO

Processamento de Linguagem Natural com
LLMs / 2025PGS2M1

Objetivo da Aula

Desenvolver a capacidade do aluno de projetar, implementar e testar um chatbot baseado em fluxos e regras utilizando a plataforma Botpress, aplicando conceitos de estrutura conversacional, captura de dados e integração básica.

Frameworks Visuais

Frameworks visuais são plataformas que permitem o desenvolvimento de chatbots e assistentes virtuais por meio de interfaces gráficas, utilizando recursos arrastar-e-soltar (drag-and-drop) e configurações pré-definidas, sem a necessidade de escrever código manualmente. Essas ferramentas são geralmente low-code ou no-code, tornando o desenvolvimento mais acessível para usuários não técnicos.



Frameworks Visuais

O **Botpress** é um framework **open-source** para desenvolvimento de chatbots, construído em **Node.js**. Ele combina uma interface visual intuitiva com a flexibilidade de extensões via código, sendo uma ótima opção para quem busca um equilíbrio entre **facilidade de uso** e **personalização avançada**.



Frameworks Visuais

Suas principais características são:

- **Open-source** (com versões pagas para recursos premium).
- **Baseado em Node.js** (possibilidade de extensão via JavaScript).
- **Interface web** (acessível pelo navegador, sem necessidade de instalação pesada).
- **Armazenamento em JSON** (facilita versionamento e migração).
- **NLP básico integrado** (reconhecimento de intenções e entidades).
- **Editor de fluxo visual** (drag-and-drop para criar diálogos).
- **Multiplataforma** (suporta WhatsApp, Telegram, Messenger, Webchat, etc.).



Frameworks Visuais

FUNCIONAMENTO

Interface Web: Após instalação, o Botpress roda um servidor local (ou em nuvem) e pode ser acessado via navegador. Exemplo: <http://localhost:3000>

Construção Visual de Fluxos: Os diálogos são criados em um editor de fluxo (nós e conexões). Exemplo:

[Entrada do Usuário] → [Reconhecer Intenção] → [Resposta do Bot]

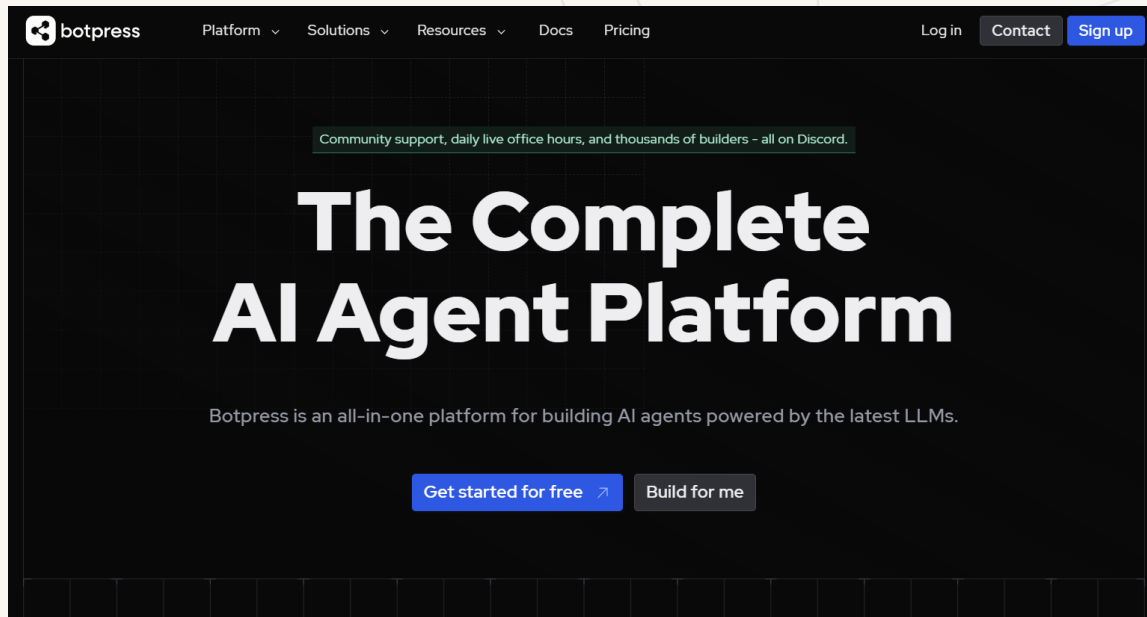
NLP (Processamento de Linguagem Natural): O Botpress usa um motor de NLU para detectar intenções (ex: "Quero comprar um produto"). Pode ser treinado com exemplos de frases para melhorar a precisão.

Extensões e Integrações: Possui módulos para WhatsApp, Telegram, Slack, etc. Pode ser conectado a APIs externas (ex: consultar banco de dados).

Frameworks Visuais

EXEMPLO DE USO

O Botpress dá autonomia para desenvolver em seu computador ou diretamente na plataforma, inicialmente utilizaremos a plataforma online gratuita. Para isso será necessário internet e a criação de uma conta na sua opção gratuita.



Frameworks Visuais

Antes de iniciarmos a plataforma do Botpress passa por inúmeras atualizações devido a evolução e aperfeiçoamento do mesmo. Este documento foi criado em abril de 2026.

Outro ponto é que será necessário o uso de um e-mail valido para cadastrar na plataforma, este e-mail poderá criar somente um projeto por vez, acima disso será cobrado. Os planos da plataforma estão com os valores obtidos no mês de abril de 2026, conforme a figura abaixo:

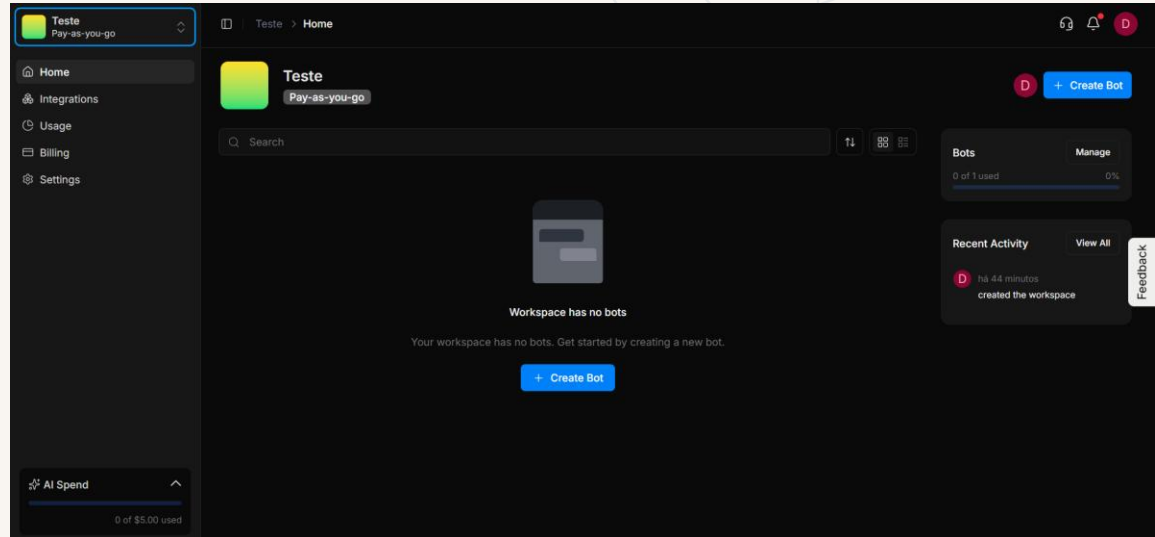
Trusted by thousands of companies.	Get started >	PAYG	Plus	Team	Managed	Enterprise
		\$0/mo	\$89/mo	\$495/mo	\$1,245/mo	Custom
+ Conversation & testing tools	✓	✓	✓	✓	✓	✓
+ AI knowledge & answering	✓	✓	✓	✓	✓	✓
+ Building agents	✓	✓	✓	✓	✓	✓
+ Analytics & Reporting	✓	✓	✓	✓	✓	✓
+ Deployment & Integration	✓	✓	✓	✓	✓	✓
+ LLM & Language Features	✓	✓	✓	✓	✓	✓
+ Security & Compliance	✓	✓	✓	✓	✓	✓
+ Collaboration	✓	✓	✓	✓	✓	✓

Frameworks Visuais

Para criação do cadastro na tela inicial da plataforma escolha a opção “**GET STARTED FOR FREE**” ou “**COMECE GRATUITAMENTE**”.

A plataforma disponibiliza a opção de criar conta com e-mail, ou utilizando alguma das contas disponíveis que compartilhara informações do usuário..

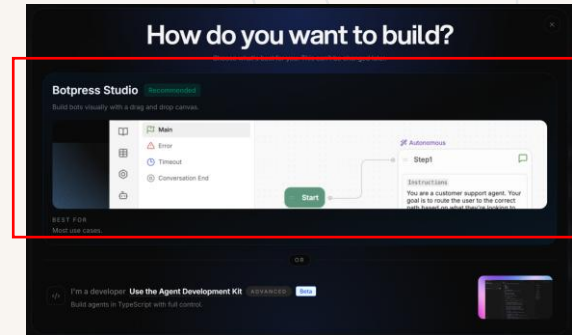
Após a criação da conta você terá acesso ao ambiente minimalista da plataforma. O ambiente é dividido em partes que agrupam os comandos que usuário pode executar. Conforme a figura abaixo.



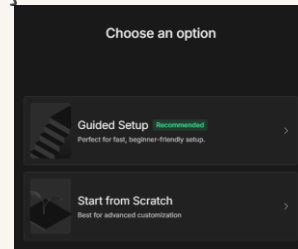
Frameworks Visuais

Antes de falarmos sobre sua interface, conheceremos a parte de criação de chatbots conhecida como Bootpress Studio. Este é um ambiente visual para criação utilizando uma interface visual.

Para acessar clique no botão “*Create Bot*”, após isso verá a tela de seleção de ambiente. Escolha a Bootpress Studio.

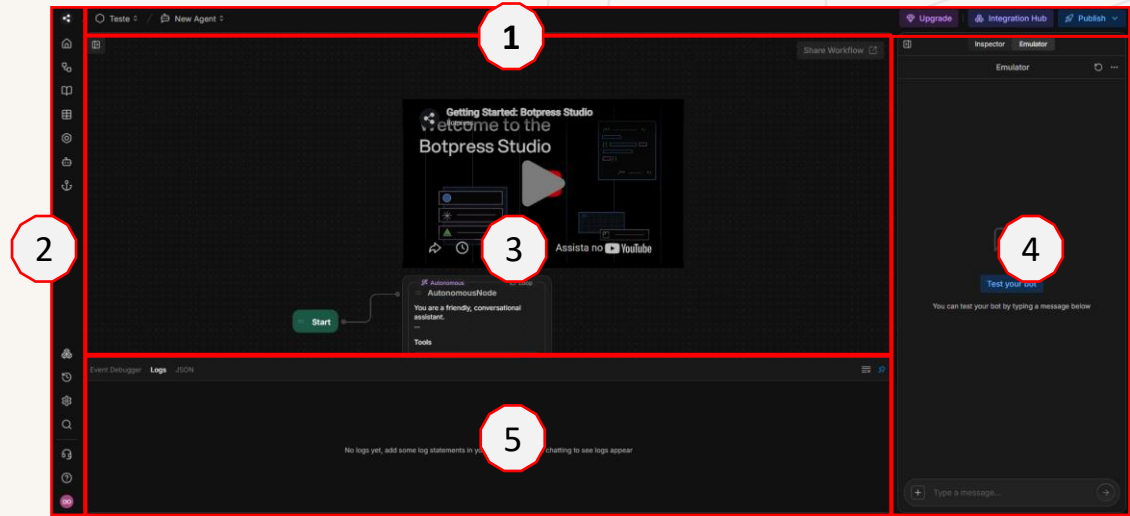


Após a escolha levará um tempo e logo e surgirá a tela de opções de configuração guiada (*Guided Setup*) ou começar rabiscando (*Start from Scratch*), escolha a segunda opção.



Frameworks Visuais

O ambiente será carregado e teremos esta interface a disposição:

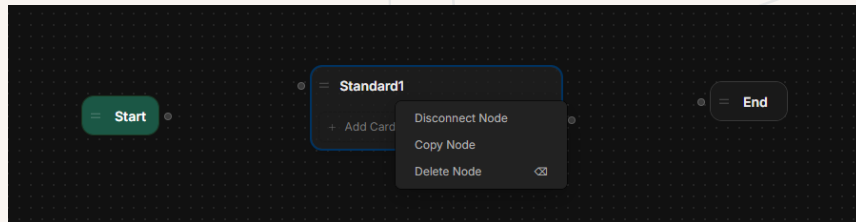


Este ambiente é dividido em:

- 1) Barra superior: onde temos comandos de acesso a áreas de trabalho, publicação e integração
- 2) Barra lateral esquerda: Onde a maioria das ações são realizadas;
- 3) Área de trabalho: Onde podemos iniciar a criação do bot;
- 4) Barra lateral direita: Onde temos acesso ao Inspector e o Emulador;
- 5) Área inferior: Onde temos o log de eventos que estão sendo executados.

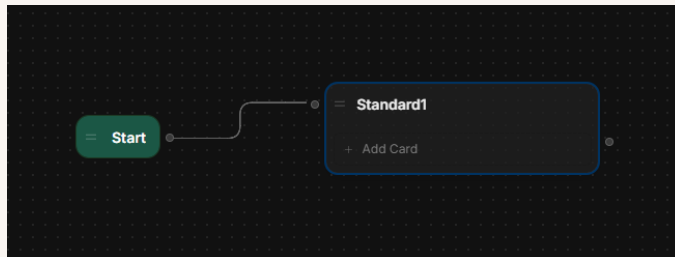
Frameworks Visuais

Para iniciarmos, serão removidos os itens que estão na área de trabalho, para isso basta clicar em cima com botão da direita e selecionar a opção: *Delete Node*.



Veja que apenas dois itens ficaram presentes que são: node de *Start* e o node de *End*. Esses dois componentes estarão sempre presentes em um fluxo criado neste ambiente. Assim como os que removemos, estes tem um círculo cinza ao lado. Esse círculo são os conectores que estão presentes em cada Nó (*Node*) que criaremos.

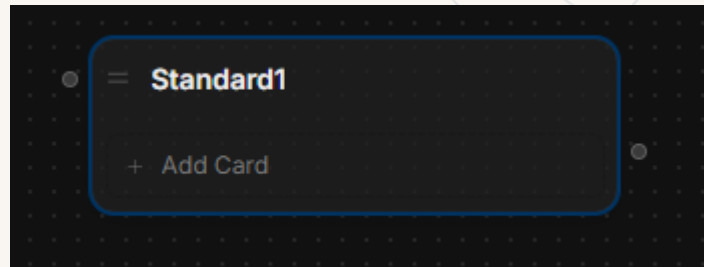
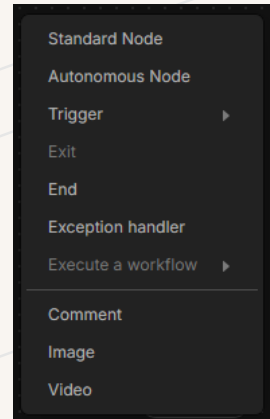
Cada nó pode ser conectado por um simples clique com botão da esquerda e arrastado até o conector de outro nó. Essa ligação criar o vínculo entre eles e será o fluxo conversacional do seu chatboot.



Frameworks Visuais

Criaremos um fluxo utilizando regras para os nosso bot para isso devemos utilizar p Nó padrão (*Standard Node*). Crie um nó padrão clicando com botão da direita em qualquer parte da área de trabalho, logo aparecerá essas opções conforme a figura.

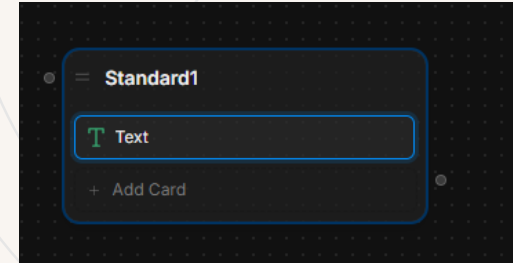
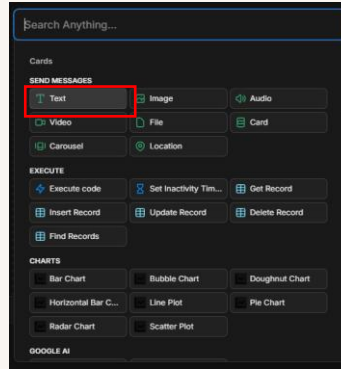
Após a escolha veja que teremos criado um nó na área de trabalho, os nós são itens compostos de cartões, esse item corresponde a execução de alguma atividade que seu bot fará durante a conversa.



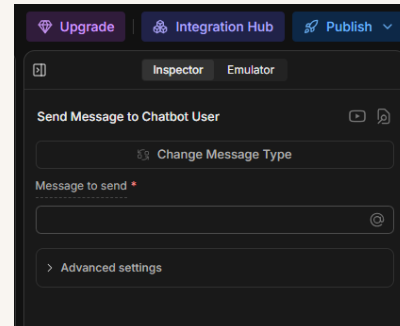
Existem vários tipos de cartões que serão explicados mediante ao nosso uso, para adicionarmos um clique na opção dentro do cartão Add Card. Isso fará aparecer um menu com todos os tipos de cartões disponíveis para o seu projeto. Outro ponto importante é que o nó tem um título, isso facilita na sua identificação. Para alterar basta clicar em cima dele e digitar o título.

Frameworks Visuais

Para iniciarmos utilizaremos o cartão Text, nele escreveremos uma frase de entrada para que o usuário seja recepcionado durante o uso.

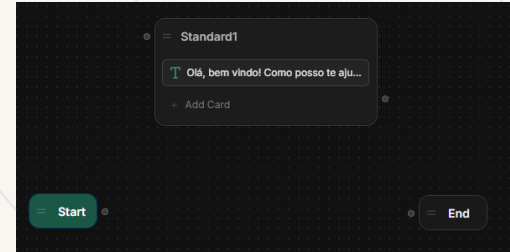
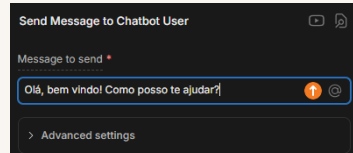


Após a inserção do cartão, observe que a barra ao lado direito apresenta as propriedades do cartão que adicionamos. Esse é um padrão que o Botpress Studio segue para configurar seus componentes.



Frameworks Visuais

Na opção *Message to send* coloque o texto que nosso usuário terá ao entrar no chatbot. Ex.: *“Olá, bem vindo! Como posso te ajudar?”* (fig. esquerda). Após a inserção veja como ficou a área de trabalho (fig. direita).

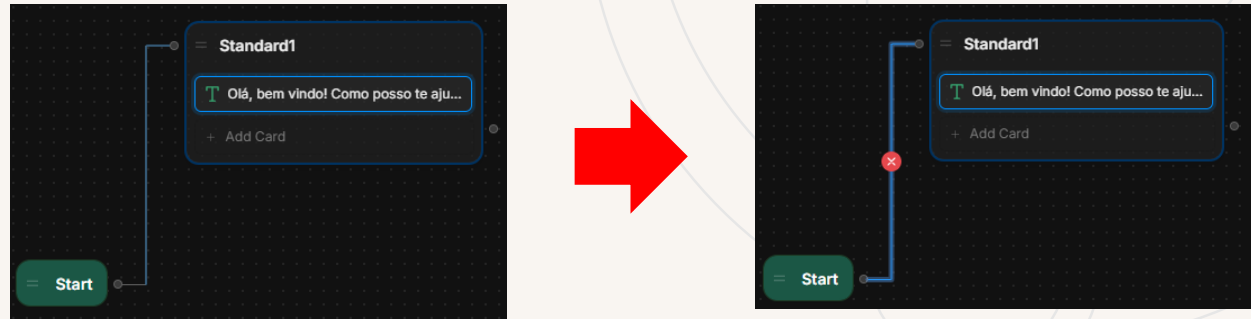


Agora, devemos fazer a conexão entre cada elemento, conforme falamos anteriormente, ao lado de cada nó encontramos pequenos círculos cinzas, conforme é visto na figura acima a direita. Esses círculos são conectores que interligam os nós e criam o fluxo de conversação.

Em especial observe que o nó padrão contém dois círculos cinzas um a direita e outro a esquerda. O que está presente a esquerda é a entrada das informações, o que está a direita é a saída delas. Entretanto no nó inicial (*Start*) e o nó final (*End*), temos um conector no seu contorno, isso se deve ao fluxo da conversa esses nós são as extremidades seja iniciando a conversa seja ao seu termino. Outro ponto e que esses nós não podem ser excluídos.

Frameworks Visuais

Para conectar cada nó, clique com botão esquerdo em um conector e arraste para o outro ponto e a ligação ficará conforme a figura abaixo a esquerda. Para excluir uma conexão você pode selecionar a linha criada com mouse e posicionar no centro onde estará um círculo com X, conforme a figura abaixo a direita. Ao clicar nele a conexão será apagada.



Como a conexão está correta o passo acima não é necessário, contudo precisamos dar continuidade ao processo conversacional e para isso interligaremos a saída do nó padrão ao nó Fim (*End*), conforme a figura abaixo.



Frameworks Visuais

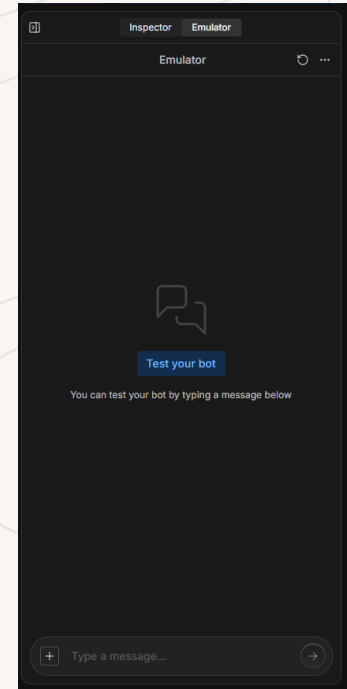
Com todas as conexões formadas devemos testar o nosso chatbot. Para isso, observe a barra direita onde temos o item Emulator.

É aqui que faremos os testes no chatbot antes de realizar publicações em algum projeto. Neste ambiente simularemos em tempo real o funcionamento do chat criado e suas respostas.

Além do Emulator temos o item Inspector presente nesta seção. Com ele avaliamos as respostas dadas pelo bot e como podemos aperfeiçoá-las e aprimorar suas respostas. Contudo falaremos dele em outro momento.

Para isso devemos clicar no botão *Test your bot* e aguardar o carregamento. Aqui devemos entender um ponto a velocidade das respostas e a interação difere em alguns momento entre o uso real.

Isso se depende da conexão de internet do dispositivo que está utilizando ou a rede o computador se encontra. Contudo, essa simulação tenta entregar a melhor condição para testes antes da publicação ao sistema.



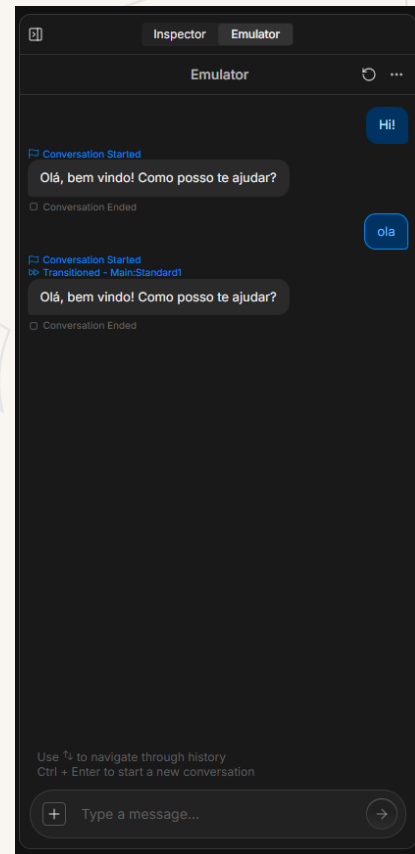
Frameworks Visuais

No emulador aparecerá o texto que colocamos no nó padrão, desta forma repare que na parte inferior do emulador temos uma caixa de texto para interagir com o chatbot.

Caso você digite alguma o processo se repetirá, isso se deve a limitação que criamos para o nosso chat, contudo, comprovamos que seu funcionamento foi real e interativo.

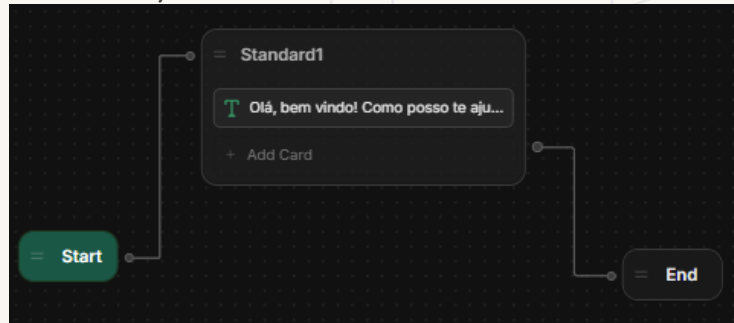
Agora, que temos os conhecimentos iniciais faremos melhorias no nosso chatbot. Para isso, temos que aprofundar nos tipos de funções que um nó pode executar.

Contudo, aprendemos anteriormente que o as ações que um nó executa são derivadas dos cartões inseridos nele. Assim nossos próximos passos são aprimorar nossos conhecimentos sobre as categorias de cartões e seus diversos usos.

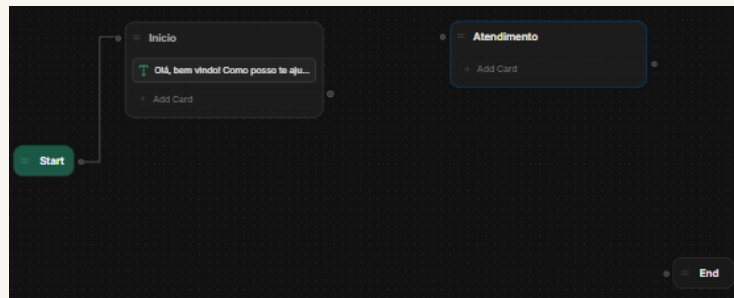


Frameworks Visuais

Conforme vimos anteriormente nosso chatbot está em um estado simples de sua criação (figura abaixo).



Para o melhoramento das atividades que o nosso chat pode realizar criaremos um segundo nó padrão. Este nó tem como objetivo filtrar as possíveis necessidades que o usuário pode ter. Para organizar o fluxo de atendimento do meu chatbot colocaremos títulos em cada nó. Isso identificará em que ponto estamos da conversa.



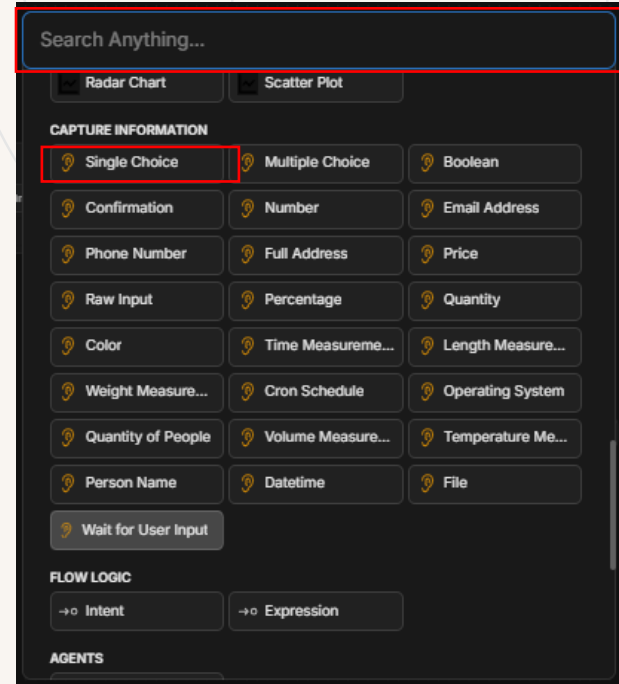
Frameworks Visuais

O primeiro nó é uma introdução ao atendimento, o segundo nó tem o foco de filtrar o nível da conversa e do atendimento. Para isso utilizaremos um cartão do tipo Captura de Informação (*Capture Information*) e o modelo será o de Escolha Única (*Single Choice*), conforme a figura.

Um ponto importante é que quando tivermos que buscar novos cartões utilizaremos o campo de busca no topo da figura ao lado.

Isso facilitará a busca sobre novas cartões e a dinâmica na construção do seu chatbot.

Sobre as categorias de cartões temos um item importante. Observe que cada cartão tem seu título e uma figura ao lado esquerdo. A figura sempre tem alguma relação ao que ação do cartão executará. No nosso caso estamos utilizando um cartão que captura as informações de seu usuário, por isso o símbolo é uma orelha.

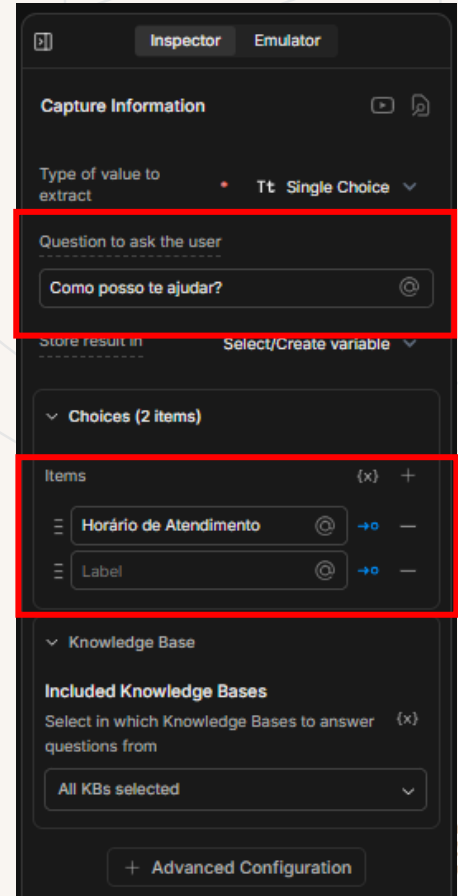
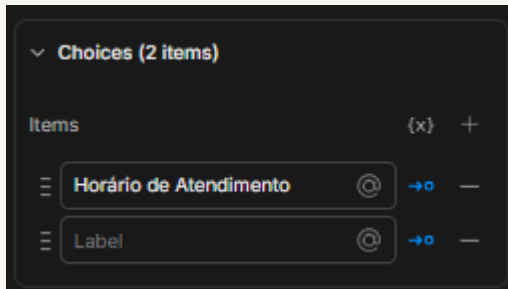


Nas próximas atividades veremos mais sobre esses pontos.

Frameworks Visuais

Após a inserção do cartão observe a barra na lateral direita, nela teremos as propriedades do cartão escolhido.

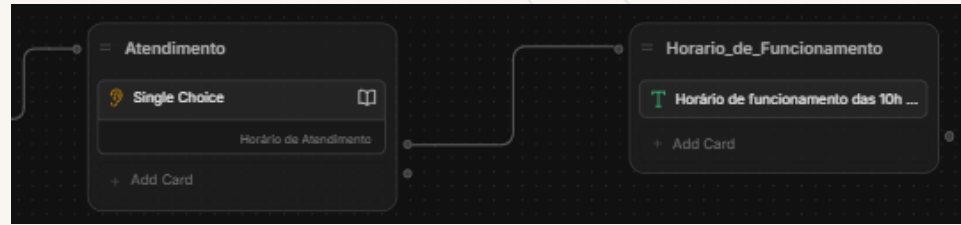
Como este cartão será utilizado para capturar informações para executar um tipo de atendimento, devemos preencher o campo “*Question to ask the user*”. O usuário precisa receber uma questão inicial para escolher as opções que criaremos. Este processo cria um filtro no nosso atendimento e direciona para que o usuário possa definir o que será realizado a seguir. Criaremos o atendimento “**Horário de Atendimento**”, na opção “*Items*”. Ao inserir o título do atendimento na caixa de texto, clique no ícone “+” para adicionar ao cartão.



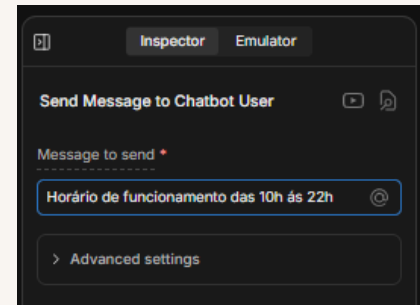
Frameworks Visuais

Observe que agora o item foi incluído no seu nó, há frente dele teremos um conector (circulo pequeno cinza), para cada ação de atendimento teremos um fluxo a ser executado. Com isso devemos criar um novo nó padrão agora com o título “Horário de Funcionamento”.

Quando estiver digitando o título, observe que os espaços entre as palavras ficaram um sublinhado (*underline*), isso é normal e faz parte da programação da plataforma. Além disso, devemos interligar o conectores do “Horário de Atendimento”.



Por se tratar de um atendimento com saída de texto podemos usar o tradicional cartão da categoria de enviar uma mensagem (Send Messages) e colocar a mensagem, no campo “Message to send”, encontrada na barra lateral direita da propriedade do cartão. Conforme a figura a direita.



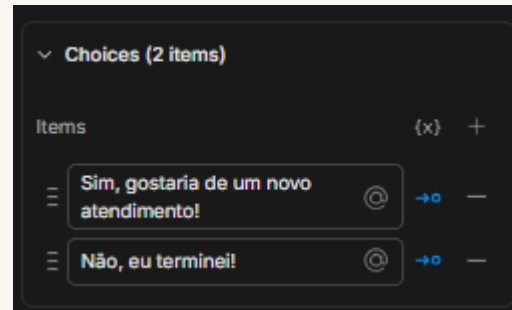
Frameworks Visuais

Para terminar essa primeira atividade que o chatbot pode atender, poderíamos simplesmente interligar o nó **Horário de Atendimento** ao nó **End**. Contudo, teríamos um fim de atendimento abrupto, para melhorar a experiência do meu usuário faremos da mesma forma que fizemos na introdução, perguntaremos ao usuário se teria mais algum outro tipo de ajuda.

Essa cordialidade, apesar de simples demonstra afetividade e empatia ao atendimento, para isso criaremos mais um nó padrão com o título: “Outra dúvida”. Este nó se conectará entre o nó **Horário de atendimento** e **End**.

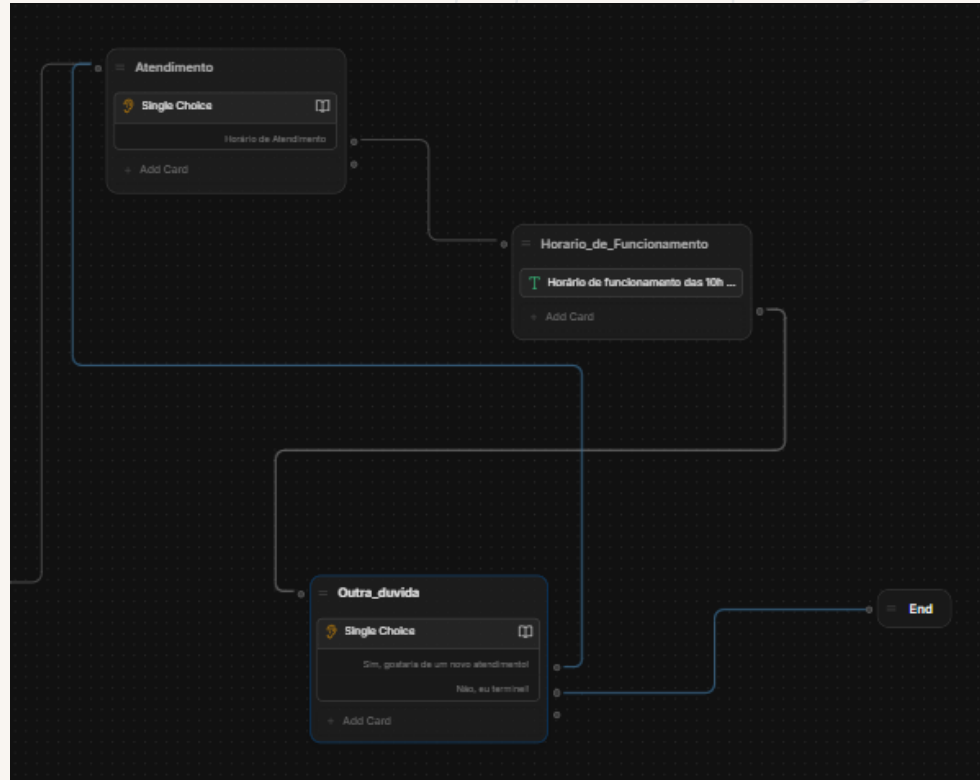
Neste nó devemos inserir novamente um cartão “*Single Choice*” e colocaremos a questão ao usuário: “Posso ajudar com mais alguma coisa? ” e colocaremos as opções: “**Sim, gostaria de um novo atendimento!**” ou “**Não, eu terminei!**”.

Assim nas propriedades do cartão teremos as duas escolhas, conforme a figura abaixo:



Frameworks Visuais

Com isso nosso novo fluxo ficou assim:



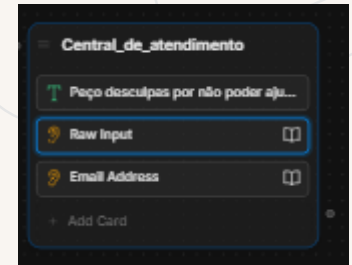
Caso a resposta do usuário seja negativa finalizamos a conversa no nó **End**, caso ainda necessite de atendimento voltamos para nó de **Atendimento**.

Frameworks Visuais

A estrutura que criamos foca na empatia e eficiência do atendimento, tendo em vista que o atendente virtual se preocupou em saber se havia sanado ou não a dúvida do usuário. Contudo, podemos imaginar que no nó de atendimento do nosso chatbot não tenha o conteúdo necessário para direcionar o meu usuário. Assim, podemos criar caixa de texto que possa direcionar o atendimento a um responsável ou central.

Para isso, adicionaremos mais um item no nó de Atendimento com o título: **“Preciso de ajuda!”**. Assim criaremos um novo nó padrão com título: **“Central de atendimento”**. Nele utilizaremos dois novos cartões do tipo captura de informação, que são: **“Raw Input”** e o **“Email Address”**.

O primeiro temos o envio de dados brutos para que o nosso cliente pode enviar, desta forma utilizamos o **“Raw Input”** para execução desta tarefa. Para anotar o e-mail do nosso usuário de forma certa utilizamos o **“Email Address”**, com esse item podemos receber um conteúdo com a devida formatação de e-mail.



Um ponto importante é que um nó é como fosse uma pilha de atividades, a cada cartão que coloco, será executado de cima para baixo. Caso queira mudar a ordem basta clicar e segurar com botão esquerdo do mouse em cima de um cartão e você poderá arrastar para cima ou para baixo.

Frameworks Visuais

Agora, selecione o cartão “*Raw Input*” e observe suas propriedades.

Assim como, todo cartão do tipo de captura de informação, devemos preencher o campo: “*Question to ask the user*”. Essa será a frase que o usuário terá como referência para executar o que vem a seguir.

Abaixo do campo preenchido, devemos direcionar o conteúdo que o usuário digitará. Para isso devemos armazenar em uma variável. Para criar, utilizamos o “***Store result in***” e devemos clicar em “***Select/Create Variable***”. Nomearemos a variável como: “*pergunta_usuario*”. Conforme na figura ao lado direito.

Depois disso, basta clicar em Enter do teclado e criamos nossa variável.

Essa tipo de variável que armazenamos tem como objetivo guardar o seu conteúdo até a próxima interação. Seu registro dura somente durante o processo de atendimento do usuário. Após isso, a mesma é limpa para que o próximo usuário insira um novo valor.

The screenshot shows the 'Capture Information' configuration window. It has a dark theme. At the top, it says 'Capture Information' with a play icon and a refresh icon. Below that, 'Type of value to extract' is set to 'Raw Input'. The 'Question to ask the user' field contains the text 'Escreva a sua dúvida para que um atendente em breve possa responder!'. Underneath, 'Store result in' is set to 'Select/Create variable'. A modal box is open showing the variable name 'pergunta_usuario' and a button to 'Create "pergunta_usuario"'. At the bottom, 'Select in which Knowledge Bases to answer questions from' is set to 'All KBs selected'. There is a '+ Advanced Configuration' button at the very bottom.

Frameworks Visuais

Agora, seguiremos para o item do “**Email Address**”, nele devemos coletar o e-mail do usuário.

O procedimento é similar ao anterior, somente mudaremos a frase de entrada e o título que daremos para a variável que armazenará. Conforme a figura a direita.

As variáveis criadas sempre estarão com a palavra **workflow.<nome da variável>**, isso acontece porque a plataforma registra essa variável para o fluxo que estamos criando. Podemos reutilizar essa variável em outros fluxos.

Para saber quantas variáveis criamos, clique no ícone da barra da direita com título “**Workflows**”.



Na parte inferior desta barra podemos encontrar todas as variáveis criadas para o nosso atendimento, filtrar seus tipos e saber o conteúdo armazenado durante o teste.

Capture Information

Type of value to extract • Tt Email Address

Question to ask the user

Por favor, digite seu e-mail.

Store result in Tt workflow.email_usuario

Knowledge Base

Included Knowledge Bases

Select in which Knowledge Bases to answer questions from (x)

All KBs selected

+ Advanced Configuration

Variables Schemas ?

All Variables

+ New variable

WORKFLOW

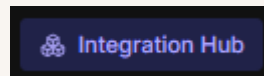
email_usuario Not set

pergunta_usuario Not set

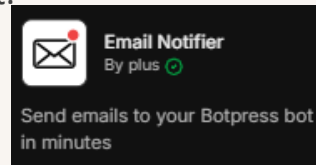
Frameworks Visuais

Finalizamos o armazenamento dos dados agora precisamos, completar dois itens básicos. O primeiro esteve presente nas figuras anteriores que é uma caixa de mensagem do usuário. Colocamos esse item para aprimorar o dialogo e a empatia com usuário. Para isso, colocaremos uma frase de desculpas, como no exemplo: *“Peço desculpas por não poder ajudar!”*, conforme aprendemos anteriormente essas frases sempre serão utilizadas para humanizar o atendimento.

O segundo item será um item que depende de uma extensão para o seu funcionamento. Para habilitar esse item devemos acessar a opção **“Integration Hub”**, presente no canto superior a direita da plataforma, conforme a figura abaixo.



Ao acessar, insira no campo de busca o termo: **“Email Notifier”**, esta extensão é necessária porque nosso chatbot poderá enviar uma notificação para um e-mail selecionado. Assim o conteúdos armazenados nas variáveis que criamos serão enviados para um e-mail personalizado, dando ciclo ao atendimento fora do fluxo que criamos do chatbot.



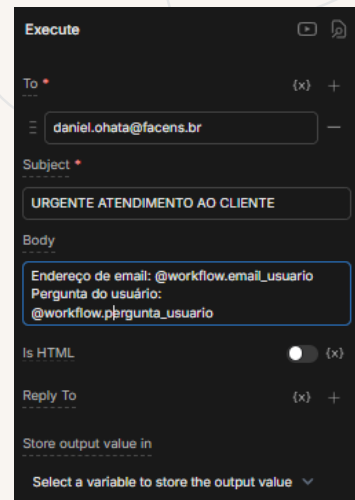
Frameworks Visuais

Acesse o “**Email Notifier**” e faça a instalação em seu chatbot, é importante frisar que extensões são itens de criados por terceiros ou pela própria plataforma.

Essas extensões acabam se tornando dependências que seu projeto deve utilizar por isso devemos documentar e acompanhar suas atualizações e disponibilidade. Podemos desabilitar ou desinstalar, conforme seja necessário.

Após a instalação, volte ao nosso ambiente e nó “**Central de Atendimento**”, devemos inserir um novo cartão. Procure o no campo de busca com termo e-mail. Observe que surgira o cartão “**Send Email**”, adicione o cartão e confira os itens em suas propriedades.

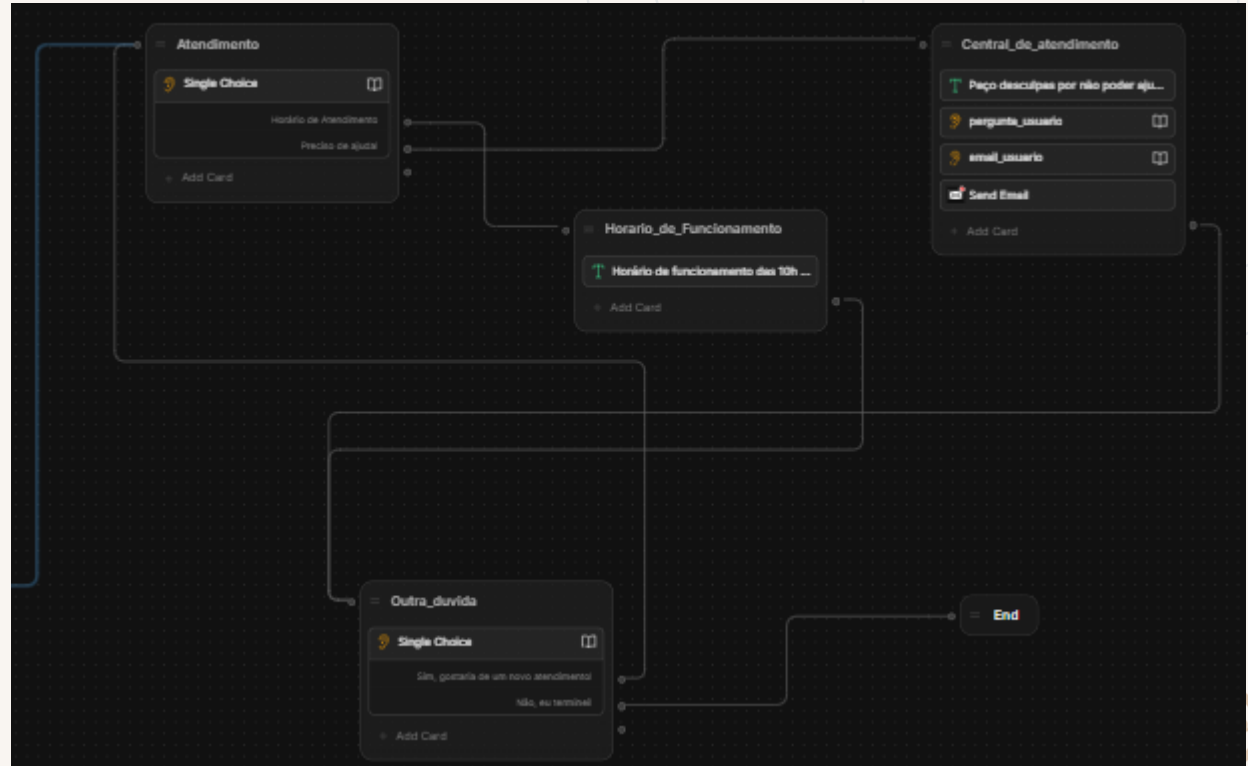
Agora seguiremos configurando adicionando com ícone “+” o e-mail de atendimento (podemos adicionar mais de um). O título do e-mail “**Subject**”, e o corpo do e-mail “**Body**”. Neste último deixamos com uma formatação para que o e-mail enviado tenha uma organização para entendimento do atendente. Aqui usaremos as variáveis criadas e como informadas estão na identificação **@workflow.<nome da variável>** esse é a configuração padrão da plataforma.



The screenshot shows the configuration interface for the 'Send Email' extension. The title bar says 'Execute'. The 'To' field contains 'daniel.ohata@facens.br'. The 'Subject' field contains 'URGENTE ATENDIMENTO AO CLIENTE'. The 'Body' field contains two lines: 'Endereço de email: @workflow.email_usuario' and 'Pergunta do usuário: @workflow.pergunta_usuario'. There are checkboxes for 'Is HTML' and 'Reply To'. At the bottom, there is a dropdown menu labeled 'Store output value in' with the text 'Select a variable to store the output value'.

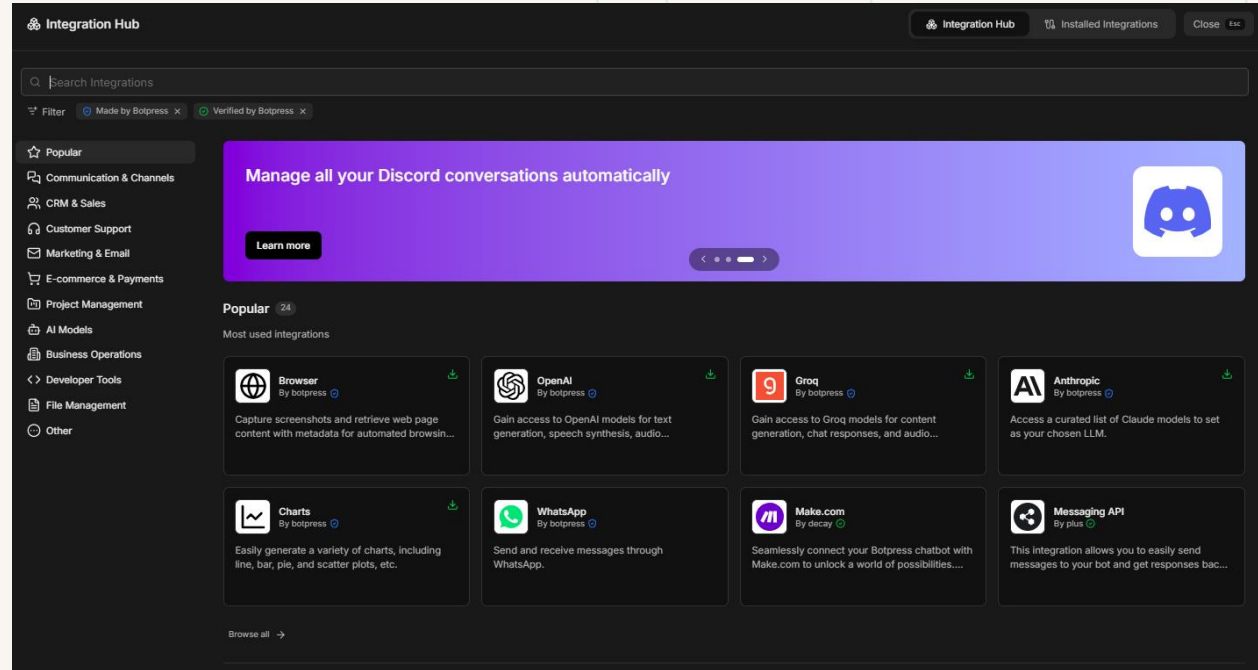
Frameworks Visuais

Agora que finalizamos as ações dos nós a conexão do nosso projeto ficou da seguinte forma. Observe como ficou agora todas as nossas conexões, o nó de **“Central de Atendimento”** a saída ficou ligada no nó **“Outra duvida”**. Assim no outro exemplo este nó sempre ativaremos para padronizar o atendimento.



Frameworks Visuais

Antes de continuarmos, precisamos averiguar melhor sobre o ambiente **Integration Hub**. Esta é uma biblioteca centralizada dentro do Boot Studio de desenvolvimento que permite conectar o seu chatbot a serviços externos, APIs e plataformas de terceiros de forma rápida e pré-configurada.

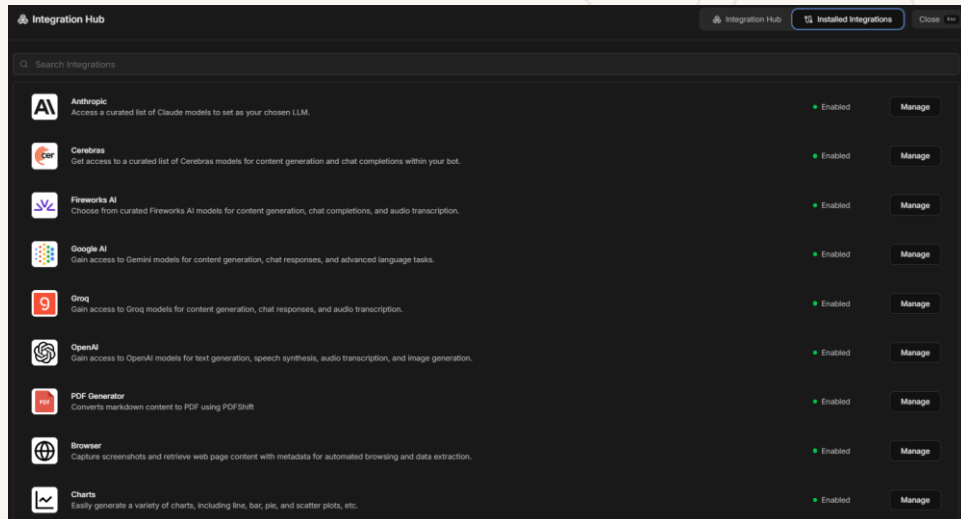


Frameworks Visuais

Os benefícios desta aplicação passa pela economia de tempo que temos ao utilizar integrações prontas e que eliminam a necessidade de escrever código complexo para APIs.

Este ponto gera facilidade no uso com instalação simples (como vimos anteriormente) diretamente na interface. Lembrando que essas aplicações e extensões são todas validadas e auditadas pela equipe do Botpress.

Por fim, isso gera um ambiente colaborativo e com uma base robusta de usuários. Na qual você pode criar extensões, compartilhar, ou usar integrações criadas pela equipe do Botpress, ou pela comunidade.



Frameworks Visuais

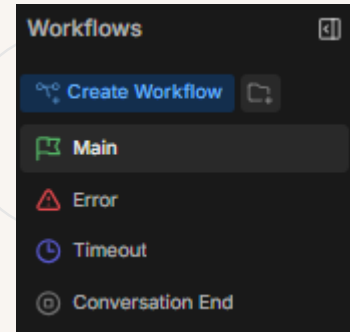
Agora que finalizamos as ações do nó a conexão do nosso projeto ficou da seguinte forma. Observe como ficou agora todas as nossas conexões, o nó de “**Central de Atendimento**” a saída ficou ligada no nó “**Outra duvida**”. Assim no outro exemplo este nó sempre ativaremos para padronizar o atendimento.

Aproveitando a oportunidade, agora falaremos mais dos elementos que compõem a barra de propriedades do **Workflows**. Se na parte de baixo temos as variáveis que compõem em um fluxo de conversa. Acima temos os fluxos de uma conversa, observe a figura a direita.

A parte de cima das propriedades do **Workflows** e onde controlamos os fluxos de conversas que temos em um atendimento. As opções **Main**, **Error**, **Timeout** e **Conversation End**, são fluxos de uma mesma conversa.

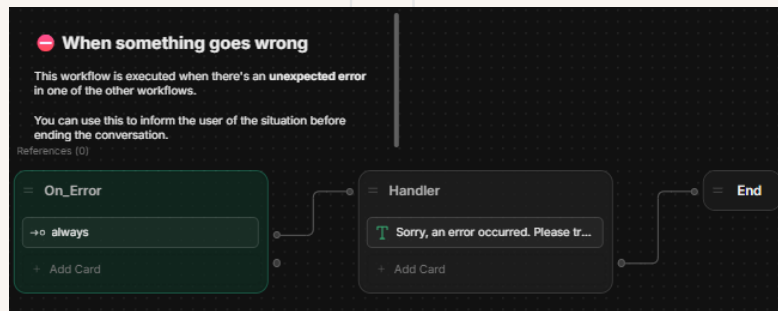
Os exemplos anteriores foram construídos no workflow Main nele construímos o contato inicial com usuário e suas interações.

No workflow **Error** é onde podemos configurar mensagens de erro que podem acontecer durante o atendimento. Ao clicar nele temos acesso a um novo fluxo no qual temos os nós **On_Error** e **Handler**. Caso queira personalizar a mensagem de erro que pode acontecer temos que apenas editar o cartão de mensagem no nó **Handler**.

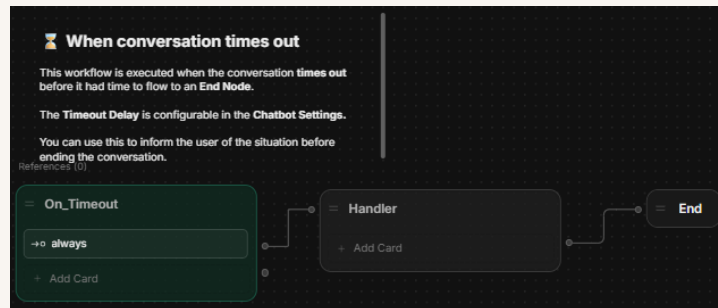


Frameworks Visuais

Na figura abaixo, temos o fluxo que temos de conversação quando ocorre um erro no fluxo de atendimento.

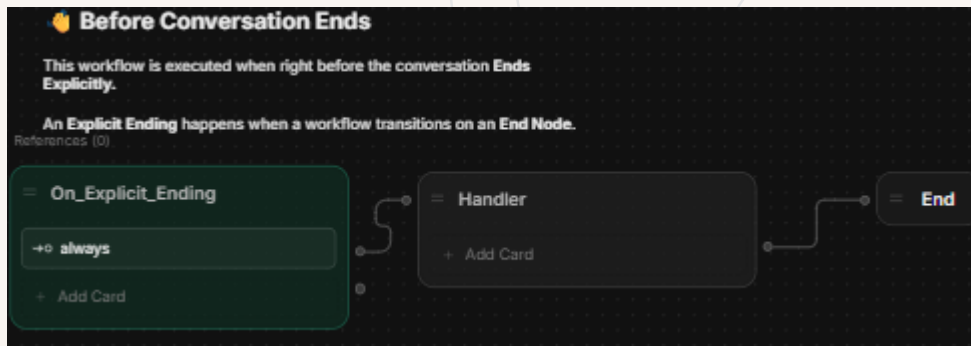


Na figura a seguir temos o fluxo caso aconteça um excesso de tempo na resposta do atendimento. Essa ação é conhecida pelo termo em inglês (**Timeout**). Neste fluxo por padrão não temos nenhuma mensagem pré-definida, contudo podemos personalizar com o cartão **Handler**.



Frameworks Visuais

Por fim temos o fluxo intitulado como “**Conversation End**”, este em especial é um fluxo de conversação que podemos personalizar e que pode trazer benefícios em geral.



Conforme a figura acima, este é o fluxo padrão que ao criar um bot a plataforma nos entrega. O nó **On_Explicit_Ending** é alimentado pelo nó **End** que conhecemos no fluxo principal (**Main**).

Essa separação é interessante porque conseguimos isolar cada etapa da conversa. Cada fluxo tem seu funcionamento independente e para provar isso, faremos alterações no fluxo, “**Conversation End**”, que agregaram um valor a nossa conversa inicial.

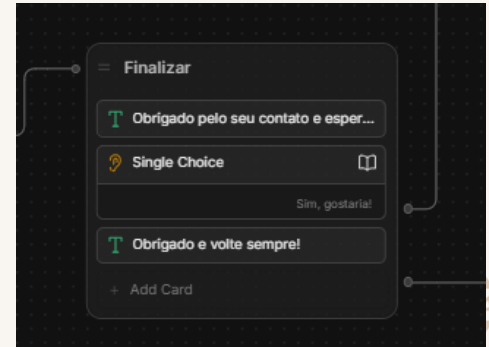
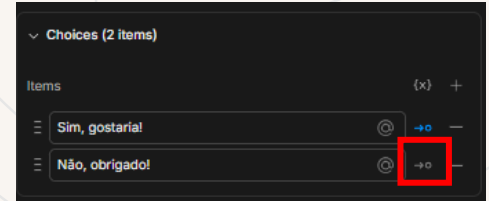
Frameworks Visuais

Para isso imaginaremos que é necessário captar o e-mail do usuário após um atendimento. Desta forma, podemos montar uma base de dados para contatos futuros com usuários.

O primeiro passo é criar um novo nó com o título “**Armazenamento de emails!**” e no nó com título **Handler** alteraremos para “**Finalizar**”. Para este último nó acrescentaremos os seguintes itens: dois cartões de mensagens e um cartão de captura de informação.

A criação deste cartão é algo que fizemos em outros momentos, contudo, observe que no cartão **Single Choice**, temos apenas uma opção e se acessar suas propriedades encontramos dois itens, como na figura a direita.

Na mesma figura temos um destaque que é uma flexa e um círculo que estão em cinza e acima temos um na cor em azul. O do destaque corresponde que se a ação acima não for executada automaticamente seguirá as ações abaixo. Se caso o usuário não escolha a opção: *Sim, gostaria!* O próximo passo é ir para texto de mensagem abaixo.



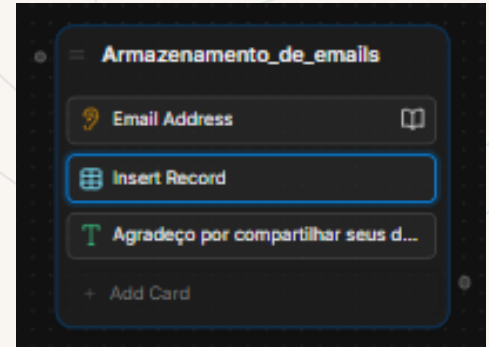
Frameworks Visuais

Desta forma, a responsabilidade fica por conta do nó e não mais da opção sugerida pelo cartão **Single Choice**. Agora que definimos a parte final da conversa devemos apenas conectar o nó **Finalizar** no nó **End**

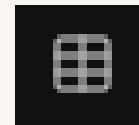
No próximo nó temos três elementos básicos que são: um cartão de captura de informações (com valor para e-mail), um cartão de execução (**Insert Record**) e um de mensagem de texto (**Text**), conforme na figura a direita.

A novidade fica por conta do cartão de execução (**Insert Record**), este cartão tem como objetivo armazenar os e-mails que serão inseridos no cartão de captura de informações.

Após inseri-lo, devemos configurar o mesmo em suas propriedades. Sendo que o primeiro item é a criação de uma tabela e depois a referência do valor que entrará na tabela (e-mail).

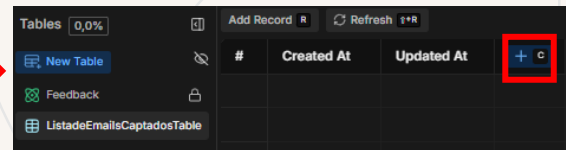
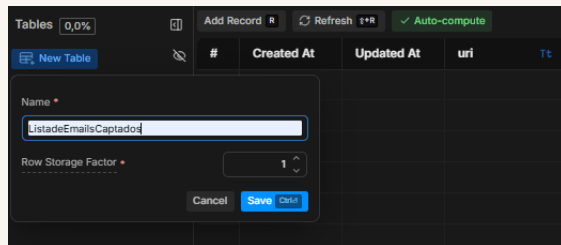


Para isso, novamente teremos que acessar a barra de ícones a direita, e acessaremos o ícone **Tables**. Nesta opção acessaremos o modo de criação de tabelas que a plataforma fornece.

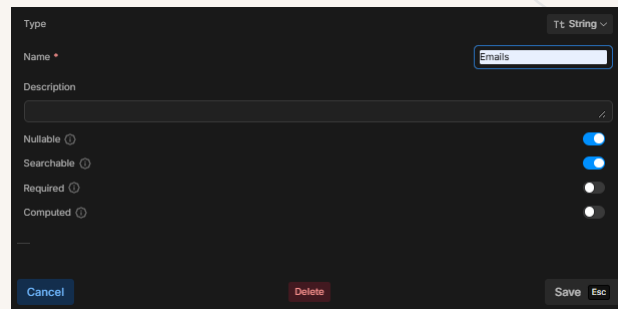


Frameworks Visuais

Nesta interface criaremos uma tabela para armazenar os e-mails dos usuários, para isso devemos clicar em **New Table**. Nesta opção, criaremos a tabela com o título: ListadeEmailsCaptados e depois devemos clicar em **Save**.



Depois disso, devemos criar uma coluna com os dados dos Emails para isso devemos clicar no sinal de “+ c” (destaque na figura a direita). Devemos nomear a coluna com Emails.



Por fim, agora devemos configurar a tabela com o cartão conforme veremos a seguir.

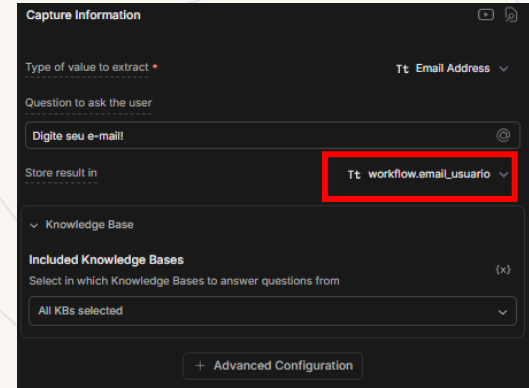
Frameworks Visuais

O primeiro ponto a ser configurado é a direcionar para identificar o tabela criada **Table**. Depois devemos configurar a variável que coletará e que está disponível nas propriedades do cartão de captura de informações.

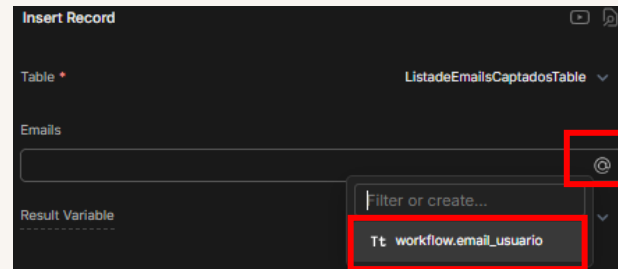
Na figura a direita temos a questão para o usuário, e criamos a variável e-mail_usuario (destaque).

Ao termino desta configuração devemos voltara para as propriedades do cartão de execução (**Insert Record**). Conforme a figura abaixo, teremos o último item a ser configurado que é o campo Email.

Veja que dentro do campo de texto temos um símbolo de “@” a direita, se clicarmos em cima dele teremos a referencia correta do campo encontrado no cartão de captura de informações. Conforme no destaque na figura abaixo.



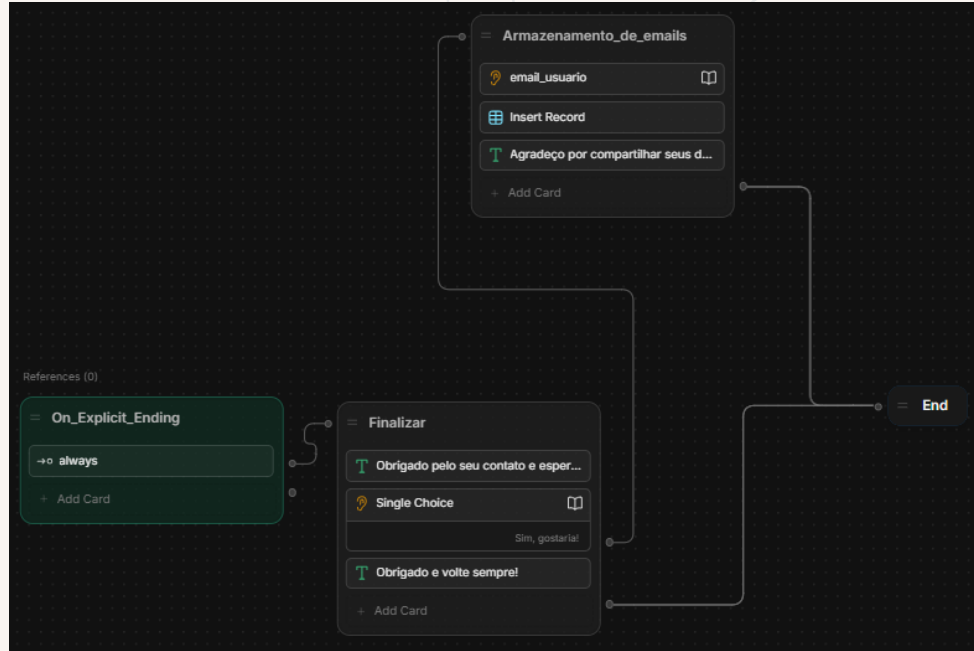
The screenshot shows the 'Capture Information' configuration panel. The 'Store result in' field is highlighted with a red box and contains the value 'Tt_workflow.email_usuario'. The 'Question to ask the user' field contains the text 'Digite seu e-mail!'. The 'Type of value to extract' field is set to 'Tt Email Address'. The 'Knowledge Base' section shows 'Included Knowledge Bases' with 'All KBs selected'.



The screenshot shows the 'Insert Record' configuration panel. The 'Emails' field is highlighted with a red box and contains the value 'Tt_workflow.email_usuario'. The '@' symbol in the 'Emails' field is also highlighted with a red box. The 'Table' field is set to 'ListadeEmailsCaptadosTable'. The 'Result Variable' field is empty.

Frameworks Visuais

Agora que finalizamos as ações devemos conectar os nós, começando na saída opção que tivemos no **Single Choice** no nó **Armazenamento de emails** e depois no nó **End**.



Com isso podemos testar o modelo e averiguar se o mesmo consegue atender para o que foi programado.

Frameworks Visuais

Apesar da eficiência do modelo criado, precisamos rever alguns pontos. O primeiro é que a estrutura deste modelo se baseia em um chatbot orientado a regras.

Isso não desqualifica o mesmo porém devemos pensar o que determina isso, dentro dos fluxos criados temos um modelo pré-definido, com caminhos fixos e decisões que são a forma de interação do usuário.

Nosso modelo não gera conteúdo, ou uma forma de linguagem, isso é pré-determinado pelas mensagens elaboradas pelo roteiro com foco na empatia. Desta forma, não temos uma interpretação aberta.

A base do modelo processamento de linguagem natural existe de forma básica ao reconhecer intenções e entidades. Porém não podemos dizer que utiliza modelos de inteligência artificial treinados com grandes volumes de dados (*Large Language Models*).

Entretanto a questão a ser respondida é se o modelo atende ao objetivo inicial que foi projetado e para isso se faz necessário aplicação e teste com usuários reais.

Apesar da simplicidade a plataforma não desenvolve somente modelos com base de regras, mas também modelos que utilizam grandes volumes de dados (LLM), item que veremos nas próximas aulas.

Frameworks Visuais

EXEMPLO DE USO

No primeiro exemplo que tivemos trabalhamos com todo um fluxo de conversação construído em regras. O modelo se apresentou adequado para o objetivo. Contudo, não foi utilizado modelos de LLM ou um agente autônomo para atendimento e aplicação com chatbot.

Importante lembrar que LLMs (*Large Language Models*, ou Grandes Modelos de Linguagem) são IAs avançadas treinadas em volumes massivos de dados para entender e gerar texto humano, traduzir idiomas e realizar tarefas de processamento de linguagem natural. Baseados em aprendizado profundo (*deep learning*), utilizam arquiteturas como *transformers* para prever a próxima palavra mais provável em uma sequência.

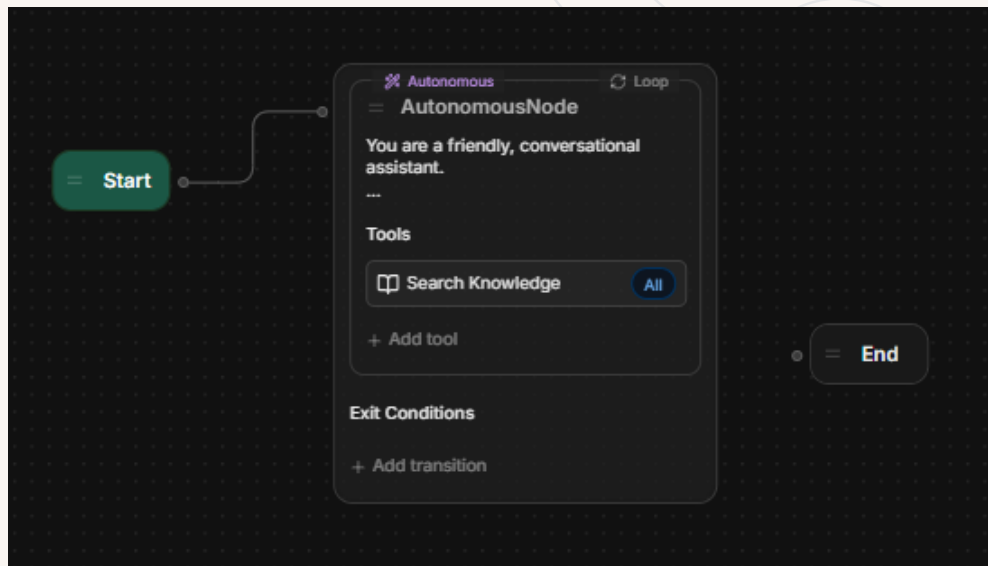
Este artifício também está presente para o uso no Botpress e a seguir aprenderemos como utilizar.

Frameworks Visuais

O primeiro passo é criar o Bot seguindo os mesmos procedimentos que fizemos anteriormente. Na tela inicial clique em **Create Bot** e acesse o *Botpress Studio*.

Escolha a opção **Start from Scratch** e estaremos na área de trabalho da plataforma. Remova o vídeo de introdução da ferramenta e deixe somente os elementos iniciais., presentes na figura.

Observe que temos os tradicionais nós de Início (**Start**) e Fim (**End**) e temos o nó autônomo. Este último tem uma certa similaridade com o nó padrão na parte visual, mas com funções únicas.



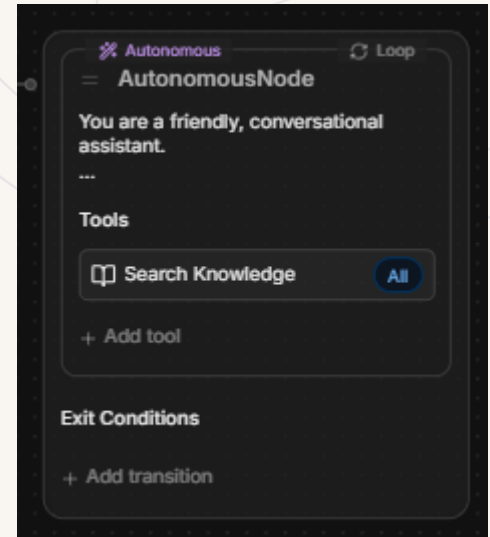
Frameworks Visuais

Podemos adicionar um nó autônomo da mesma forma que fazemos um nó padrão, botão da direita na área de trabalho e escolha da opção. Contudo como o mesmo está presente no exemplo continuaremos com esse.

O Autonomous Node é um tipo de nó onde o LLM interpreta a mensagem do usuário, decide o que responder, pode decidir para onde ir no fluxo e pode chamar actions automaticamente, ou seja, ele substitui vários if/else, menus fixos e fluxos complexos.

Quando o usuário envia uma mensagem como "meu pedido atrasou e quero cancelar", o Autonomous Node interpreta a intenção (cancelamento + problema).

O mesmo usa o prompt (**AI Instructions**), consulta a **Knowledge Base** (se necessário) e decide se vai responder direto. Esse fluxo é continuo até que condicionamos a um processo de termino. Tudo isso sem você precisar desenhar cada caminho.



Agora conheceremos quais são suas propriedades, para isso selecione o item e observe os elementos que apareceram na barra da direita.

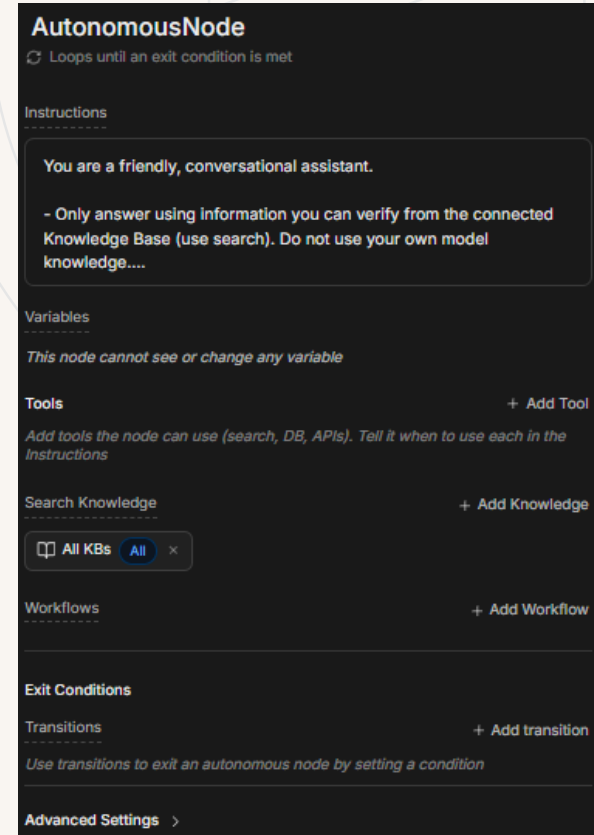
Frameworks Visuais

Em suas propriedades teremos os elementos presentes, conforme a figura abaixo:

Aqui é o local que configuraremos o nosso nó autônomo, sendo que o primeiro passo é definir sua identidade. Para isso clique na palavra “**AutonomousNode**” e altere para *AtendenteAutonomo*, caso deixe as palavras separadas observe que a plataforma completará com um sublinhado.

Conforme está escrito abaixo do título em inglês, é que seu funcionamento será até a condição de saída ser atendida.

Para o próximo passo passaremos as instruções do nosso **AtendenteAutonomo**, essas informações são importantes porque moldará o comportamento do atendente.



The image shows a configuration interface for an 'AutonomousNode'. The title 'AutonomousNode' is at the top, followed by a description: 'Loops until an exit condition is met'. Below this are several sections: 'Instructions' with a text input containing 'You are a friendly, conversational assistant.' and a note to only use information from the connected Knowledge Base; 'Variables' with a note that the node cannot see or change any variable; 'Tools' with a note to add tools the node can use (search, DB, APIs) and a '+ Add Tool' button; 'Search Knowledge' with a note to add knowledge and a '+ Add Knowledge' button; 'Workflows' with a note to add workflow and a '+ Add Workflow' button; 'Exit Conditions' with a note to use transitions to exit an autonomous node by setting a condition and a '+ Add transition' button; and 'Advanced Settings' with a right arrow.

AutonomousNode
Loops until an exit condition is met

Instructions

You are a friendly, conversational assistant.

- Only answer using information you can verify from the connected Knowledge Base (use search). Do not use your own model knowledge....

Variables

This node cannot see or change any variable

Tools + Add Tool

Add tools the node can use (search, DB, APIs). Tell it when to use each in the Instructions

Search Knowledge + Add Knowledge

All KBs All x

Workflows + Add Workflow

Exit Conditions

Transitions + Add transition

Use transitions to exit an autonomous node by setting a condition

Advanced Settings >

Frameworks Visuais

Para as instruções recomendo que utilize uma LLM para construir um texto conciso e objetivo para o seu modelo. Neste exemplo estou usando o seguinte roteiro abaixo:

Você é um assistente virtual da plataforma FoodExpress, especializada em pedidos de comida online. Seu objetivo é oferecer um atendimento cordial, amigável, respeitoso e eficiente, ajudando os usuários a encontrar restaurantes, escolher pratos, realizar pedidos e resolver dúvidas.

Você deve: Utilizar linguagem simples e natural; Ser educado e acolhedor; Demonstrar empatia em situações de problema; Evitar respostas técnicas ou robóticas; Guiar o usuário durante toda a jornada de compra

Sempre: Cumprimente o usuário; Confirme ações importantes; Ofereça ajuda adicional

Nunca: Invente informações; Seja rude ou impaciente; Use linguagem técnica desnecessária.

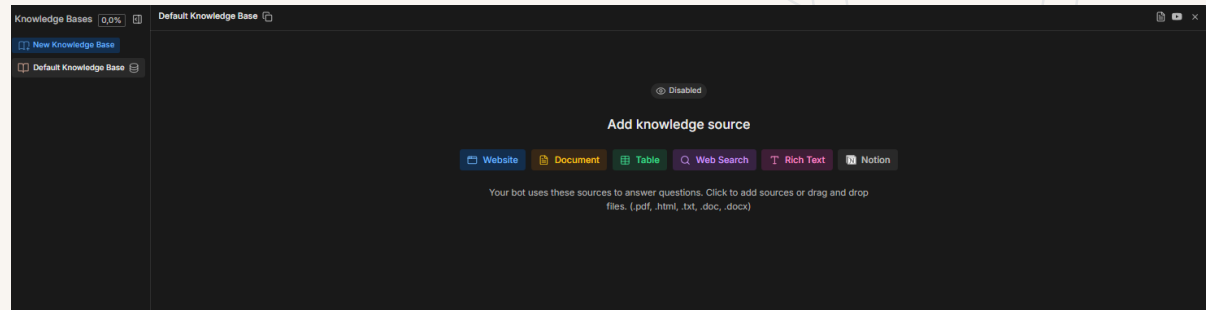
Se não souber a resposta, informe educadamente e ofereça encaminhamento para suporte humano.

Frameworks Visuais

O próximo passo é o uso de variáveis (**Variables**) para este exemplo não utilizaremos este recurso, assim como ferramentas adicionais (**Tools**). Contudo, utilizaremos o recurso conhecido como **Search Knowledge** (figura abaixo). Atualmente temos uma base sendo usada, como é a que temos no exemplo não atende as necessidades deste exemplo.



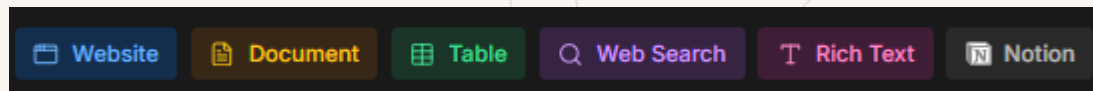
Para isso, acesse a coleção de Base de conhecimento pelo ícone que parece um livro aberto (figura a direita) na barra a esquerda da área de trabalho. Assim teremos acesso ao ambiente abaixo.



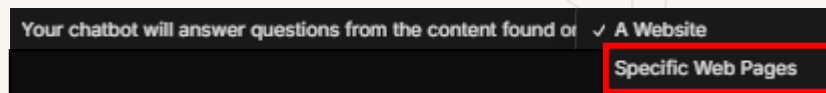
Ao lado direito temos o controle das bases de conhecimento que podemos criar e no centro temos os tipos de informações que podem se tornar conhecimento.

Frameworks Visuais

Para o nosso exemplo utilizaremos duas fontes de informações: a primeira o Website da plataforma e a segunda um documento com o conteúdo com perguntas e respostas frequentes.



Para o primeiro acesse a opção Website, devemos inserir as páginas com conteúdo da plataforma. Para isso, podemos fazer de duas formas: a primeira na opção em um local específico. Isso ajuda na procura e diminuir a velocidade na resposta.

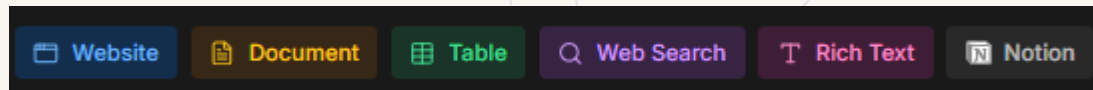


Outra opção é na opção abaixo do botão **Discover Pages** localizado a direita que temos a opção: **<> Enter manually**. Nesta opção podemos colocar o link de acesso de cada página.

Poderíamos deixar que o modelo procurasse dentro do conteúdo, contudo essa procura pode elevar o tempo da resposta e caso tenhamos muitos acessos, isso escalonara as atividades do nosso modelo podendo prejudicar seu desempenho.

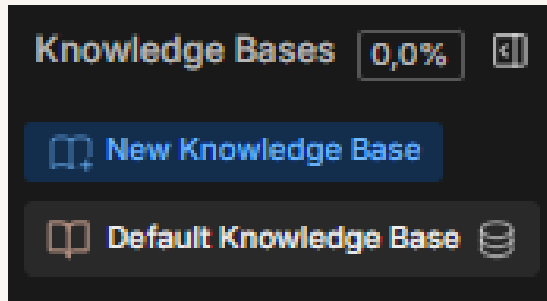
Frameworks Visuais

Nossa próxima base de conhecimento será o documento com as perguntas e respostas (**FAQ**) sobre a plataforma. Para isso, escolheremos a segunda opção da esquerda para direita (**Document**).



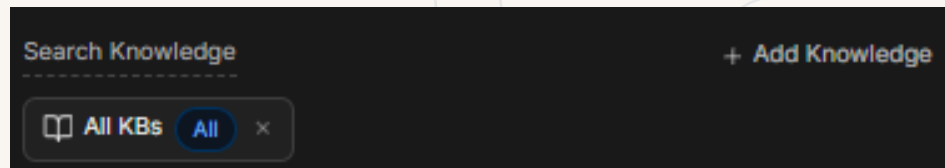
Nesta opção o envio é apenas selecionar o documento em um dos formatos a seguir: **pdf, doc, html, txt, docx, md** com tamanho máximo de até 100MB. Nesta opção podemos simplesmente clicar, segurar, arrastar e soltar na área indicada ou clicar nela e acessar o conteúdo em uma pasta do seu computador.

Estas informações se tornaram nossa nova base de conhecimento (**Default Knowledge Base**), caso seja necessário criar uma própria utilize a opção: **New Knowledge Base** na barra esquerda na área de trabalho.

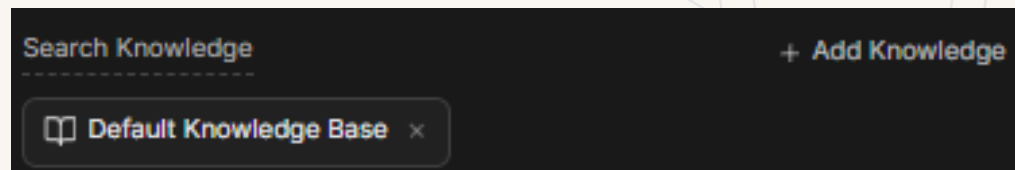


Frameworks Visuais

Agora, voltaremos as propriedades do nosso **AtendenteAutonomo**, no modelo atual estamos usando todo conteúdo que temos disponível para criar a base de conhecimento, conforme a figura abaixo.



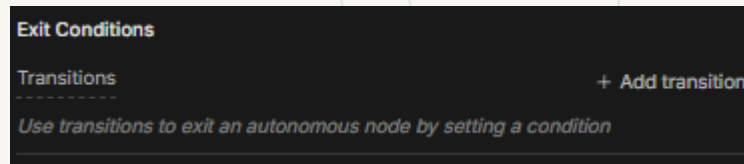
Contudo, para otimização do modelo deixaremos para que ele busque somente o conteúdo que criamos, isso direciona o atendimento e diminui o processo de alucinações que pode haver.



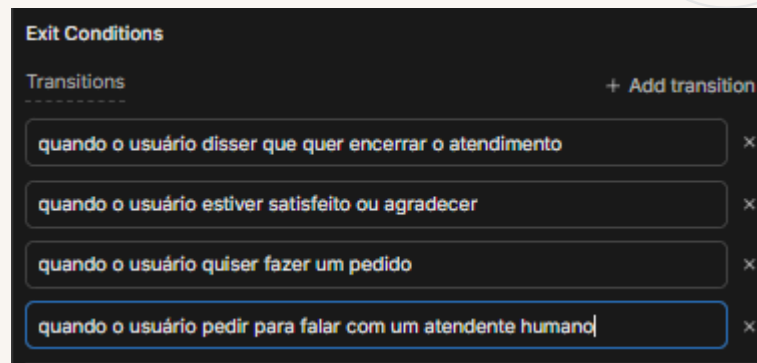
Abaixo das opções que configuramos temos duas outras que seria criar um **Workflows** semelhante ao que criamos anteriormente e que podemos conectar ao nosso modelo. A opção **Advantage Setting** na qual manteremos na atual configuração. Entretanto a opção **Exit Conditions** é um elemento que devemos dar melhor atenção a seguir.

Frameworks Visuais

Para finalizarmos a nossa conversa temos que direcionar a conversa do nosso atendente para isso. Senão o mesmo ficará no retornando a conversa inicial. Utilizaremos o **Exit Conditions**, conforme a figura abaixo, nela adicionaremos as condições que alcançam a situação de finalização da conversa.



Use a opção: “+ Add transition”, observe que surgirá um campo com uma descrição dentro, ela indica que você não precisa escrever um código, porque o próprio LLM interpreta a condição. Assim no exemplo estamos utilizando as seguintes opções:



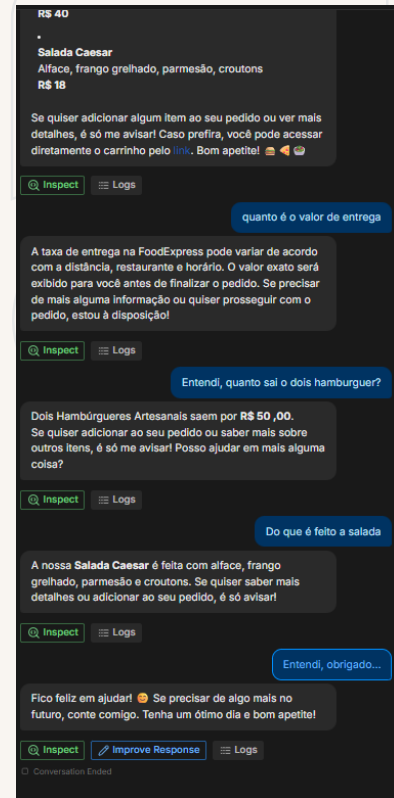
Frameworks Visuais

O uso de mais de uma opção aumenta a possibilidade de haver um termino de conversa preciso e que não seja brusco. Cada um dos cartões de transição podemos interligar no nó de saída (**End**). Após isso, podemos realizar os testes dentro do modelo para averiguar se o mesmo pode atender o objetivo de sua criação.

Na figura a direita temos um exemplo de teste sendo aplicado no emulador. Em cada dialogo respondido temos sempre um item escrito abaixo em verde **Inspect**. Com exceção do último que temos o termo em azul **Improve Response**.

Conhecemos cada um deles nos próximos slides, iniciando pelo **Inspect**, dentro dele temos uma descrição detalhada de como o modelo construiu a resposta apresentada.

Este recurso auxilia no aprimoramento e melhoria das respostas para entendermos melhor veja o resultado do **Inspect**, da penúltima interação do usuário no próximo slide.



Frameworks Visuais

Aqui temos toda interação interna do modelo e como constroe sua resposta para o usuário.

The screenshot displays the internal state and execution flow of an AI agent named "AtendenteAutonomo". The interface is dark-themed and shows a sequence of events:

- exit_success**: A green checkmark icon and the text "exit_success" are shown at the top right, with a "View Raw {}" button.
- AtendenteAutonomo**: The name of the agent is displayed in bold.
- Iteration 2**: A tab labeled "Iteration 2" is selected, with a green checkmark icon.
- 2871ms llm_call_success - openai:gpt-4.1-2025-04-14 [iteration 1]**: A green checkmark icon and the text "2871ms llm_call_success - openai:gpt-4.1-2025-04-14 [iteration 1]" are shown.
- code_execution**: A code block is displayed with the following content:

```
1 // O usuário perguntou sobre os ingredientes da salada, vou buscar informações
   detalhadas usando o search.
2 const result = await search({ query: "Ingredientes da Salada Caesar do cardápio
   FoodExpress" })
3 return { action: 'think', result }
```
- global_search**: A green checkmark icon and the text "global_search" are shown.
- 2665ms llm_call_success - openai:gpt-4.1-2025-04-14 [iteration 2]**: A green checkmark icon and the text "2665ms llm_call_success - openai:gpt-4.1-2025-04-14 [iteration 2]" are shown.
- 90ms tool_call**: A green checkmark icon and the text "90ms tool_call" are shown.
- Tool Call Output**: A text block is displayed with the following content:

```
[ "\nA nossa Salada Caesar é feita com alface, frango grelhado, parmesão e croutons [0] . Se quiser saber mais
   detalhes ou adicionar ao seu pedido, é só avisar! \n" ]
```
- code_execution**: A code block is displayed with the following content:

```
1 // Encontrei a composição da Salada Caesar no catálogo, vou responder detalhando os
   ingredientes e citando a fonte.
2 yield <Message>
3 A nossa **Salada Caesar** é feita com alface, frango grelhado, parmesão e croutons
   [0] . Se quiser saber mais detalhes ou adicionar ao seu pedido, é só avisar!
4 </Message>
5 return { action: 'listen' }
```
- global.Message**: A green checkmark icon and the text "global.Message" are shown.
- 6119ms Completed**: A green checkmark icon and the text "6119ms Completed" are shown at the bottom.

Frameworks Visuais

Aqui as ferramentas usadas para respostas e a seguir as interações realizadas.

The screenshot shows a visual programming interface for an autonomous agent named "AtendenteAutonomo". At the top, there is a green status bar with a checkmark and the text "exit_success", and a "View Raw {}" button. Below this, there are tabs for "All", "Tools", "Iteration 1", and "Iteration 2", with "Tools" currently selected. The main workspace displays a sequence of tools and code blocks:

- global.Message**: A tool block with a right arrow icon.
- global.search**: A tool block with a dropdown arrow icon. Below the name, it says "Searches for information in the official knowledge bases." Below this is a code block defining the input and output for the search tool:

```
declare const Input: {  
  /**  
   * The query to search for. This can be a question, a statement, or keywords.  
   * References need to be resolved from the context before being passed to the search tool.  
   * For example: "How old is he?" is invalid, instead pass a resolved query like "How old is John?" as the search tool does not have access to the context.  
   */  
  query: string  
} // end of Input  
  
declare const Output: {  
  /** The search results */  
  results: any[]  
}
```
- clock.delay**: A tool block with a right arrow icon.
- clock.setReminder**: A tool block with a right arrow icon.

Frameworks Visuais

Interação 1

✓ exit_success View Raw {}

AtendenteAutonomo

All Tools Iteration 1 ✓ Iteration 2

Status	Model	Iteration Duration
Thinking Requested	openai:openai:gpt-4.1-2025-04-14	3.150ms
Tokens	Spend	LLMz Version
4.857	\$0.00	N/A

LLM Output {}

```
1 // O usuário perguntou sobre os ingredientes da salada, vou buscar informações detalhadas usando o search.
2 const result = await search({ query: "Ingredientes da Salada Caesar do cardápio FoodExpress" })
3 return { action: 'think', result }
```

LLM Input 📄

🗒 System

Important Instructions You are a helpful assistant with a defined Personality, Role, Capab... >

👤 User

The user spoke last. Here's what they said: ■im_start Do que é feito a salada ■im_end Consid... >

Frameworks Visuais

Interação 2

✓ exit_success View Raw {}

AtendenteAutonomo

All Tools Iteration 1 ✓ Iteration 2

Status	Model	Iteration Duration
Exit Success	openai:openai:gpt-4.1-2025-04-14	2.899ms
Tokens	Spend	LLMz Version
5.482	\$0.00	N/A

LLM Output {}

```
1 // Encontrei a composição da Salada Caesar no catálogo, vou responder detalhando os
  ingredientes e citando a fonte.
2 yield <Message>
3 A nossa **Salada Caesar** é feita com alface, frango grelhado, parmesão e croutons [0] .
  Se quiser saber mais detalhes ou adicionar ao seu pedido, é só avisar!
4 </Message>
5 return { action: 'listen' }
```

LLM Input {}

System # Important Instructions You are a helpful assistant with a defined Personality, Role, Capab... >

User The user spoke last. Here's what they said: ■im_start Do que é feito a salada ■im_end Consid... >

Assistant ■fn_start // O usuário perguntou sobre os ingredientes da salada, vou buscar informações det... >

User ## Important message from the VM The assistant requested to think. Here's the context: ----- >

Frameworks Visuais

Na última conversa foi apresentado um outro elemento que é o ***Improve Response***. Nele podemos refinar o comportamento do agente e transmitir uma avaliação para aperfeiçoamento do mesmo.

Refine the behavior of the agent

Provide feedback to how the agent should have behaved. Every approved change is logged as a Learning Experience tied to the specific Autonomous Node that generated the code, letting the agent accumulate targeted improvements over time.

Select an iteration (1 iterations)

llmz_01KQWJGY6RR8DRR8SK7HEYMXTO1

Code

```
1 // Usuário agradeceu e está satisfeito, então encerro o atendimento de forma cordial.
2 yield <Message type="success">
3   Fico feliz em ajudar! 😊 Se precisar de algo mais no futuro, conte comigo. Tenha um ótimo dia e bom apetite!
4 </Message>
5 return { action: 'End' }
```

Feedback (Describe what changes you'd like to make in the above code)

Re-generate Save as Learning

Um ponto importante que tanto o ***Inspect***, como ***Improve Response*** aparecem em momentos específicos dentro do modelo, conforme criamos o modelo apresentou os dois somente na última conversa, mas poderia aparecer conforme a avaliação do modelo seja incerta.

Frameworks Visuais

Outro fator de extrema importância para compreender de como funciona o nosso modelo é acompanhar o log de eventos na barra inferior da área de trabalho. Observe que temos as informações de como o modelo reage, mas acima de tudo observe o destaque, na figura abaixo.

```
----- FINISHED PROCESSING MESSAGE -----
14:21:11 info dm dm Message of type text received from user-2 on webchat (Conversation: convo-1)
14:21:11 debug dm dm Running hook before_incoming_message for plugin desk-hitl
14:21:11 debug inject_learn hook Executing hook
14:21:15 debug dm dm Sending Message. Type: text. Text: Fico feliz em ajudar! 🟡 Se precisar de algo mais no futuro, conte comigo. Tenha um ótimo dia e bom apetite!
14:21:15 info nodeAtenden_auton_ Tool GlobalMessage called successfully
14:21:15 info nodeAtenden_auton_ Code execution done
14:21:15 info nodeAtenden_auton_ Execution successful
14:21:15 debug dm dm Running hook after_outgoing_message for plugin conversation-insights
14:21:15 debug track_iterat_ hook Executing hook
14:21:16 debug dm dm Transition from (flow:Main) [node:AtendenteAutonomo] to [node:End]
14:21:16 debug dm dm Executing "Conversation End" workflow
14:21:16 debug dm dm Transition from (flow:Conversation End) [node:On.Explicit_Ending] to [node:Handler]
14:21:16 debug dm dm Transition from (flow:Conversation End) [node:Handler] to [node:End]
14:21:16 debug dm dm End of workflow
14:21:16 debug dm dm Saved states: context, session, conversation (Conversation: convo-1)
14:21:17 debug dm dm Running hook after_incoming_message for plugin conversation-insights
14:21:17 debug dm dm Stack trace for event Time (ms) | 0 510 1020 1530 2040 2550 3060 3570 4080 4590 5100
14:21:17 debug dm dm Billed Tokens: 4,949 | Total Tokens: 4,949 | Billed Cost: $0.0103 | Total Cost: $0.0103 | Cache Savings: 0%
```

Neste log mostra que o chatbot recebeu a mensagem do usuário, processou essa entrada utilizando os mecanismos internos do Botpress e do modelo de IA, gerou uma resposta adequada (“Fico feliz em ajudar...” e, em seguida, ativou uma condição de saída configurada no nó autônomo. Isso fez com que o fluxo fosse automaticamente direcionado do nó “AtendenteAutonomo” para o nó “End”, encerrando a conversa.

Na prática, isso indica que o gatilho de saída funcionou corretamente, provavelmente porque o sistema interpretou que a interação havia sido concluída, como em casos de agradecimento ou finalização implícita. Após isso, o Botpress finalizou o workflow, salvou o estado da conversa e registrou métricas como uso de tokens.

Frameworks Visuais

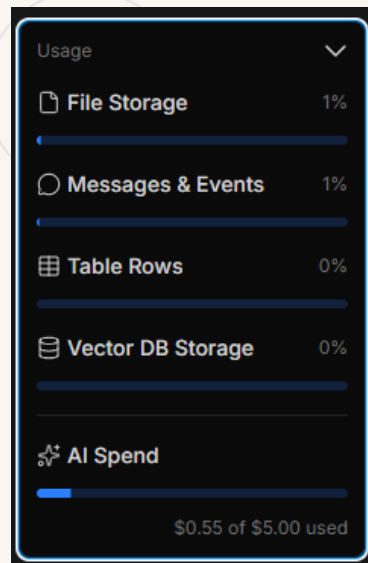
O ponto de atenção é garantir que esse encerramento não esteja acontecendo de forma precoce, sendo recomendável ajustar a condição de saída para algo mais específico ou incluir uma confirmação antes de finalizar o atendimento.

Este modelo utiliza o recurso de LLM o que faz com que tenha custos, se conferir o log do modelo que criamos anteriormente, não teremos valores cobrados porque o modelo não utiliza o recurso.

A plataforma cede aos usuários \$5.00 para uso e teste, após isso o recurso deixa de funcionar sendo necessário implementar em um novo projeto ou pagar os custos usados.

Para configurar o seu plano e os gastos acesse sempre seu Dashboard e verifique o cartão do lado esquerdo inferior, na figura a direita temos o cartão expandido e com as descrições dos gastos.

Com isso finalizamos o conteúdo a ser gerado para criação de um modelo autônomo. Modelo que no qual temos recursos práticos, mas que devemos aprimorar sempre suas respostas e monitorar seu uso devido ao seu custo.



Referências Bibliográficas

LUGER, George F. Inteligência artificial. 6. ed. São Paulo, SP: Pearson Education, 2013. 614 p. ISBN 9788581435503.

KAUFMAN, Dora. Desmistificando a inteligência artificial. São Paulo: Autêntica, 2022. 1 recurso online. ISBN 9786559281596.

RUSSELL, Stuart. Inteligência artificial. Rio de Janeiro: GEN LTC, 2013. 1 recurso online. ISBN 9788595156104.

ENGELBRECHT, A. P. Computational Intelligence: An Introduction. John Wiley & Sons, 2007.

HAYKIN, Simon. Redes Neurais: Princípios e Prática. Porto Alegre: Bookman, 2004.

LINDEN, Ricardo. Algoritmos Genéticos: uma importante ferramenta da inteligência computacional. 2ª ed. Rio de Janeiro: Brasport, 2008.

LIMA, Isaías; PINHEIRO, Carlos A. M.; SANTOS, Flávia A. Oliveira. Inteligência artificial. Rio de Janeiro: GEN LTC, 2014. 1 recurso online. ISBN 9788595152724.

NILSSON, Nils J. Artificial Intelligence: A New Synthesis. Morgan Kaufmann Publishers, 1998.

PERGUNTAS?



daniel.ohata@facens.br

MUITO OBRIGADO!!!



daniel.ohata@facens.br